MHOUI-Miner: Exact and Scalable Mining of Maximal High-Occupancy Utility Itemsets via Array-Projected Galois Closure Leaps and Pareto-Trie Pruning

Quan Van[0009−0002−8222−0076]

Independent Researcher, Ho Chi Minh City, Vietnam
vanhaminhquan2406@gmail.com

Abstract. High-Utility Itemset Mining (HUIM) and High-Occupancy Itemset Mining (HOIM) uncover orthogonal facets of knowledge in discrete data systems: economic monetary yield versus physical transactional compactness. Existing unified frameworks (e.g., HUOPM) aggregate both dimensions into a composite ratio, conflating physical basket coverage with monetary return and creating the high-value dilution paradox. Furthermore, unconstrained mining inevitably suffers from combinatorial pattern explosion. In this paper, we formulate the problem of mining **Maximal High-Occupancy Utility Itemsets** ($\mathcal{MHOUI}_{\mathrm{s}}$), which decouples classical transaction occupancy $|X|/|T|$ from total economic utility $u(X)$ and enforces strict superset Pareto maximality: an itemset is retained if and only if no proper superset achieves both equal-or-higher occupancy and equal-or-higher utility. To prevent exhaustive brute-force candidate enumeration and overcome the non-monotonic nature of both occupancy and utility, we establish a tree-structured set-enumeration architecture powered by three foundational pruning theorems: (1) a tight **Joint Dual Upper Bound** ($\mathrm{UB}_{\mathrm{joint}}$) simultaneously bounding extension utility and occupancy envelopes in single-pass linear scans; (2) **Galois Support-Closure Leaps** jumping directly across equivalence classes with exact utility invariance, bypassing exponential interior subtree branches; and (3) an inverted rare-item index (**Pareto-Trie**) enabling $O(1)$ branch dominance pruning against previously materialized maximal frontiers. We propose MHOUI-Miner, an exact in-process algorithm built upon flattened array projection and 64-bit vector bitsets. Extensive empirical evaluations across standard retail and transactional benchmark databases (including million-scale datasets such as Chainstore) confirm that MHOUI-Miner prunes up to 98.4% of visited search nodes, finishes in sub-second execution speeds, sustains memory under 450 MB, and compresses pattern volume by up to 99% without losing actionable Pareto-optimal business intelligence.

Keywords: High-utility itemset mining · High-occupancy itemsets · Maximal pattern mining · Pareto dominance · In-process pruning · Set-enumeration tree.

1 Introduction

Frequent itemset mining (FIM) constitutes an indispensable pillar in knowledge discovery and intelligent information systems [1,16,37,32]. Given a transaction database, FIM extracts all item combinations whose occurrence frequency satisfies a user-specified minimum support threshold (σ). However, frequency alone often fails to reflect the qualitative value or the structural density of the discovered patterns. On the one hand, an itemset may appear in numerous transactions while accounting for only a trivial fraction of items within each transaction. To address this limitation, high-occupancy itemset (HOI) mining was introduced [28,38,6,5,22,35]. Classical transaction occupancy, defined as $|X|/|T|$, measures the completeness with which an itemset X covers a supporting transaction T.

On the other hand, traditional FIM treats all items uniformly, disregarding differences in economic profit, item quantity, semantic importance, or risk. High-utility itemset mining (HUIM) addresses this concern by incorporating internal quantities and external unit profits [21,20,9,39,11,34,18,27,17,4,29,23,36,13,33]. Recent studies have explored high utility-occupancy pattern mining (HUOPM) [12,3,7,19], which computes the ratio of pattern utility to total transaction utility, $u(X,T)/TU(T)$. While effective for proportional profit analysis, HUOPM couples utility and occupancy into a single composite metric, obscuring whether a pattern has high physical basket coverage or merely high unit cost.

Moreover, mining all unconstrained patterns inevitably leads to the *pattern explosion problem*. To achieve concise representations, maximal frequent itemset (MFI) mining discards frequent subsets dominated by larger frequent supersets [2,14,15]. Similarly, maximal and closed high-utility itemsets compress utility collections [30,8,10,26,31]. However, directly applying support-based or utility-based maximality to occupancy-constrained patterns is fundamentally flawed: neither classical occupancy nor total utility satisfies the anti-monotonicity property. A superset can exhibit higher or lower occupancy and utility than its subsets.

To overcome these foundational challenges, this paper formulates the **Maximal High-Occupancy Utility Itemset** ($\mathcal{MHOUI}_{\mathrm{s}}$) mining problem and introduces MHOUI-Miner. The proposed framework simultaneously enforces:

1. **Support Constraint:** The itemset must occur in at least σ transactions.
2. **Classical Occupancy Constraint:** The itemset must satisfy a minimum average transaction occupancy β, ensuring dense physical coverage.
3. **Total Utility Constraint:** The itemset must generate at least μ economic utility.
4. **Superset Pareto Maximality:** An itemset is eliminated if there exists a strict superset possessing equal or higher occupancy and equal or higher utility.

Structural Search Space: Set-Enumeration Tree vs. General Lattice Graph. A central algorithmic design decision in combinatorial pattern mining lies in choosing between traversing a general hypergraph/lattice or spanning a

canonical set-enumeration tree [24]. While a general lattice graph represents all subset-superset containment relations directly, it suffers from severe duplicate node generation: an itemset of size k can be reached through $k!$ distinct search paths, necessitating complex visited-set hashing or expensive synchronization in memory. In sharp contrast, a canonical lexicographical set-enumeration tree guarantees that every itemset is assigned a unique generator prefix, eliminating duplicate traversal paths by construction. However, standard set-enumeration trees risk visiting an exponential number of dead-end candidate prefixes ($O(2^m)$) if search bounds are loose.

In MHOUI-Miner, we solve this challenge by establishing an exact, in-process tree traversal that minimizes visited nodes through three non-trivial pruning mechanisms:

1. *Coupled Dual Upper Bounding:* Rather than computing expensive permutation sorts at every tree node, we formulate the **Joint Dual Upper Bound** ($\mathrm{UB}_{\mathrm{joint}}$) evaluated in $O(|\Gamma(P)|)$ linear single-pass scans.
2. *Galois Support-Closure Leaps:* Instead of exploring itemsets one by one, MHOUI-Miner detects support equivalence in $O(1)$ bitwise operations, performing Galois closure leaps $P \to P \cup \{e\}$ that skip entire intermediate tree levels without loss of information.
3. *Inverted Pareto-Trie Dominance:* We maintain an inverted rare-item index (**Pareto-Trie**) that conducts superset dominance checks in real time, cutting off dead-end subtrees before recursive allocation.

2 Related Work and Positioning

Table 1 rigorously positions $\mathcal{MHOUI}_{\mathrm{s}}$ relative to established pattern mining paradigms.

Table 1. Positioning of $\mathcal{MHOUI}_{\mathrm{s}}$ against related itemset mining paradigms.

Pattern Family	Support	Classical Occ.	Total Utility	Utility Occ.	Maximality Criterion
FI [1,16,37,32]	✓	–	–	–	None
MFI [2,14,15]	✓	–	–	–	Support-maximal
HOI [28,38,6,5,22,35]	✓	✓	–	–	None
HUI [21,20,9,39,11,34]	Optional	–	✓	–	None
MaxHUI / CHUI [30,8,10,26,31]	Optional	–	✓	–	Utility-based concise form
HUOPM [12,3,19]	✓	–	Indirect	✓	None
MaxCloFHUOIM [7]	✓	–	Indirect	✓	Composite utility-occupancy maximality
HOUI (Unconstrained)	✓	✓	✓	–	None
Weak MHOUI	✓	✓	✓	–	Weak Superset Pareto Dominance
Strong MHOUI (MHOUI-Miner)	✓	✓	✓	–	Strong Superset Pareto Dominance

Critical Distinction Between Decoupled MHOUI and HUOPM/MaxCloFHUOIM. The foundational advance of this work lies in decoupling physical transactional occupancy from total monetary return:

– *The High-Value Dilution Paradox in HUOPM:* In the HUOPM formulation introduced by Gan et al. [12] and recently extended to concise representations

by Duong et al. [7] (MaxCloFHUOIM), occupancy is defined as the utility ratio $u(X,T)/TU(T)$. While meaningful for measuring proportional profit share, this formulation creates a severe distortion: an itemset comprising a single extremely expensive luxury item (e.g., a jewelry piece generating \$2,000 in a \$2,010 basket containing 20 items) achieves an inflated utility occupancy of 99.5%, despite physically accounting for merely 5% of the transaction items (1/20). Conversely, a complete product bundle occupying 100% of a physical transaction with modest margin is discarded if high-revenue background goods are present elsewhere.

– *Decoupled Multi-Objective Pareto Formulation:* $\mathcal{MHOUI}_{\mathrm{s}}$ treats physical completeness ($|X|/|T| \geq \beta$) and absolute economic yield ($u(X) \geq \mu$) as orthogonal objective dimensions. An itemset is eliminated if and only if there exists a strict superset that dominates it in *both* dimensions. This guarantees that decision-makers receive compact, highly representative, and economically actionable pattern frontiers without distortion.

3 Preliminaries and Problem Formulation

Let $\mathcal{I} = \{i_1, i_2, \ldots, i_m\}$ be a finite set of distinct items. A quantitative transaction database is $\mathcal{D} = \{T_1, T_2, \ldots, T_n\}$. Each transaction $T_q \in \mathcal{D}$ consists of an itemset $T_q \subseteq \mathcal{I}$, an internal quantity $q(i, T_q) \in \mathbb{Z}^+$ ($q(i, T_q) \geq 1$) denoting the purchase count of item $i \in T_q$, and an external unit profit $p(i) \in \mathbb{R}_{>0}$ ($p(i) > 0$). Consequently, all item utilities $u(i, T_q) = p(i) \cdot q(i, T_q) > 0$ are strictly positive.

Definition 1 (Item and Transaction Utility). *The utility of an item i in transaction T_q is $u(i, T_q) = p(i) \cdot q(i, T_q)$. The utility of an itemset $X \subseteq T_q$ in T_q is $u(X, T_q) = \sum_{i \in X} u(i, T_q)$. The transaction utility of T_q is $TU(T_q) = \sum_{i \in T_q} u(i, T_q)$.*

Definition 2 (Support and Database Utility). *The supporting transaction set of itemset X is $\Gamma(X) = \{T_q \in \mathcal{D} \mid X \subseteq T_q\}$. The support count of X is $\mathrm{supp}(X) = |\Gamma(X)|$. The total utility of X in $\mathcal{D}$ is $u(X) = \sum_{T_q \in \Gamma(X)} u(X, T_q)$.*

Definition 3 (Classical Transaction Occupancy). *For an itemset $X \subseteq T_q$, the classical occupancy of X in T_q is $\mathrm{occ}(X, T_q) = \frac{|X|}{|T_q|}$. The average classical occupancy of X across its supporting transactions is:*

$$\mathrm{aocc}(X) = \frac{1}{\mathrm{supp}(X)} \sum_{T_q \in \Gamma(X)} \mathrm{occ}(X, T_q) = \frac{|X|}{\mathrm{supp}(X)} \sum_{T_q \in \Gamma(X)} \frac{1}{|T_q|}. \tag{1}$$

Definition 4 (Transaction-Weighted Utilization). *The transaction-weighted utilization (*TWU*) of an itemset X is:*

$$\mathrm{TWU}(X) = \sum_{T_q \in \Gamma(X)} TU(T_q). \tag{2}$$

Definition 5 (High-Occupancy Utility Itemset ($\mathcal{HOUI}$)). *Given a minimum support threshold $\sigma \in \mathbb{N}^+$, a minimum occupancy threshold $\beta \in (0, 1]$, and a minimum utility threshold $\mu \in \mathbb{R}^+$, an itemset X is a High-Occupancy Utility Itemset if:*

$$\text{supp}(X) \geq \sigma, \quad \text{aocc}(X) \geq \beta, \quad \textit{and} \quad u(X) \geq \mu. \tag{3}$$

3.1 Superset Pareto Dominance and MHOUI Variants

Definition 6 (Weak Superset Dominance). *An itemset Y weakly dominates X (denoted $X \prec_{\text{w}} Y$) if $X \subset Y$, $\text{aocc}(Y) \geq \text{aocc}(X)$, $u(Y) \geq u(X)$, and at least one inequality is strict ($\text{aocc}(Y) > \text{aocc}(X)$ or $u(Y) > u(X)$).*

Definition 7 (Strong Superset Dominance). *An itemset Y strongly dominates X (denoted $X \prec_{\text{s}} Y$) if $X \subset Y$, $\text{aocc}(Y) \geq \text{aocc}(X)$, and $u(Y) \geq u(X)$.*

Definition 8 (Weak and Strong MHOUI). *An itemset $X \in \mathcal{HOUI}$ is a **Weak MHOUI** ($\mathcal{MHOUI}_{\text{w}}$) if $\nexists Y \in \mathcal{HOUI}$ such that $X \prec_{\text{w}} Y$. An itemset $X \in \mathcal{HOUI}$ is a **Strong MHOUI** ($\mathcal{MHOUI}_{\text{s}}$) if $\nexists Y \in \mathcal{HOUI}$ such that $X \prec_{\text{s}} Y$.*

Theorem 1 (Inclusion Chain). *For any database $\mathcal{D}$ and thresholds σ, β, μ: $\mathcal{MHOUI}_{\text{s}}(\mathcal{D}, \sigma, \beta, \mu) \subseteq \mathcal{MHOUI}_{\text{w}}(\mathcal{D}, \sigma, \beta, \mu) \subseteq \mathcal{HOUI}(\mathcal{D}, \sigma, \beta, \mu)$.*

Proof. If $X \in \mathcal{MHOUI}_{\text{s}}$, no $Y \in \mathcal{HOUI}$ satisfies $X \prec_{\text{s}} Y$. Since $X \prec_{\text{w}} Y \implies X \prec_{\text{s}} Y$, no such Y can weakly dominate X. Thus $X \in \mathcal{MHOUI}_{\text{w}} \subseteq \mathcal{HOUI}$.

Mathematical Distinction Between Weak and Strong Dominance: The fundamental distinction lies in whether equality in both objective dimensions permits elimination:

- *Weak Superset Dominance ($X \prec_{\text{w}} Y$):* Requires at least one strict inequality ($\text{aocc}(Y) > \text{aocc}(X)$ or $u(Y) > u(X)$). If a proper superset $Y \supset X$ exhibits identical metrics ($\text{aocc}(Y) = \text{aocc}(X)$ and $u(Y) = u(X)$), Y does *not* weakly dominate X. Consequently, $\mathcal{MHOUI}_{\text{w}}$ retains both X and Y.
- *Strong Superset Dominance ($X \prec_{\text{s}} Y$):* Admits non-strict inequalities ($\text{aocc}(Y) \geq \text{aocc}(X)$ and $u(Y) \geq u(X)$). Even if metrics are exactly tied, the superset Y subsumes X because Y represents a larger, more comprehensive pattern without any sacrifice in occupancy or monetary yield. Hence, $\mathcal{MHOUI}_{\text{s}}$ eliminates X and retains only Y.

For instance, if $X = \{a, b\}$ has $\text{aocc}(X) = 0.60, u(X) = \$100$ and superset $Y = \{a, b, c\}$ has $\text{aocc}(Y) = 0.60, u(Y) = \$100$ (e.g., c generates zero marginal profit or co-occurs identically in the same transaction set), Weak MHOUI outputs both patterns ($\{X, Y\}$), whereas Strong MHOUI outputs only the maximal superset $\{Y\}$.

3.2 Running Example Database and Detailed Metric Calculations

To provide concrete intuition for all theoretical definitions, bounding theorems, and search mechanics, we introduce a running toy quantitative transaction database $\mathcal{D}_{\text{toy}}$ comprising 5 transactions and 5 distinct items $\mathcal{I} = \{a, b, c, d, e\}$, shown in Table 2. Table 3 presents the corresponding external unit profit table.

Table 2. Running quantitative transaction database $\mathcal{D}_{\text{toy}}$ with transaction lengths and utilities.

TID	**Items with Quantities**	**Item Utilities** ($u(i, T_q)$)	**Length** ($\lvert T_q \rvert$)	**TU** ($TU(T_q)$)
T_1	$(a:1), (b:2), (c:1)$	$u(a) = 3, u(b) = 2, u(c) = 5$	3	\$10
T_2	$(a:2), (c:1), (d:1)$	$u(a) = 6, u(c) = 5, u(d) = 2$	3	\$13
T_3	$(a:1), (b:3), (c:2), (e:1)$	$u(a) = 3, u(b) = 3, u(c) = 10, u(e) = 4$	4	\$20
T_4	$(b:1), (c:1), (d:2)$	$u(b) = 1, u(c) = 5, u(d) = 4$	3	\$10
T_5	$(a:1), (b:1), (c:1), (d:1)$	$u(a) = 3, u(b) = 1, u(c) = 5, u(d) = 2$	4	\$11
Total Database Utility $\sum_{T_q \in \mathcal{D}_{\text{toy}}} TU(T_q)$:				\$64

Table 3. External unit profit table for items in $\mathcal{D}_{\text{toy}}$.

Item (i)	a	b	c	d	e
Unit Profit ($p(i)$)	\$3	\$1	\$5	\$2	\$4

Example 1 (Utility and Transaction Utility Calculation). In transaction T_1, item a has quantity $q(a, T_1) = 1$, yielding utility $u(a, T_1) = 1 \times \$3 = \$3$. Item b has $q(b, T_1) = 2 \implies u(b, T_1) = 2 \times \$1 = \$2$. Item c has $q(c, T_1) = 1 \implies u(c, T_1) = 1 \times \$5 = \$5$. The transaction utility of T_1 is $TU(T_1) = 3 + 2 + 5 = \$10$. Across the database, the supporting transactions of itemset $\{a, c\}$ are $\Gamma(\{a, c\}) = \{T_1, T_2, T_3, T_5\}$. Its total database utility is:

$$u(\{a, c\}) = (3 + 5) + (6 + 5) + (3 + 10) + (3 + 5) = 8 + 11 + 13 + 8 = \$40.$$

Example 2 (Classical Physical Occupancy vs. Utility Occupancy). In transaction T_1, the classical physical occupancy of $\{a, c\}$ is $\text{occ}(\{a, c\}, T_1) = |\{a, c\}|/|T_1| = 2/3 \approx 0.6667$. Across all supporting transactions $\Gamma(\{a, c\}) = \{T_1, T_2, T_3, T_5\}$, its average physical occupancy is:

$$\text{aocc}(\{a, c\}) = \frac{|\{a, c\}|}{\text{supp}(\{a, c\})} \sum_{T_q \in \Gamma(\{a,c\})} \frac{1}{|T_q|} = \frac{2}{4}\left(\frac{1}{3} + \frac{1}{3} + \frac{1}{4} + \frac{1}{4}\right) = \frac{2}{4} \cdot \frac{7}{6} = \frac{7}{12} \approx 0.5833.$$

In sharp contrast, the utility occupancy of $\{a, c\}$ in T_1 would be $u(\{a, c\}, T_1)/TU(T_1) = \$8/\$10 = 0.8000$, illustrating that physical basket coverage (66.7%) and monetary share (80.0%) represent distinct semantic dimensions.

Example 3 (Transaction-Weighted Utilization and Item Filtering). Let user-specified thresholds be minimum support $\sigma = 3$, minimum occupancy $\beta = 0.40$ (40%), and minimum utility $\mu = \$20$. Computing $\text{supp}(i)$ and $\text{TWU}(i)$ for all items $i \in \mathcal{I}$:

- Item a: $\Gamma(a) = \{T_1, T_2, T_3, T_5\} \implies \text{supp}(a) = 4 \geq 3$, $\text{TWU}(a) = 10 + 13 + 20 + 11 = \$54 \geq 20$.
- Item b: $\Gamma(b) = \{T_1, T_3, T_4, T_5\} \implies \text{supp}(b) = 4 \geq 3$, $\text{TWU}(b) = 10 + 20 + 10 + 11 = \$51 \geq 20$.
- Item c: $\Gamma(c) = \{T_1, T_2, T_3, T_4, T_5\} \implies \text{supp}(c) = 5 \geq 3$, $\text{TWU}(c) = 10 + 13 + 20 + 10 + 11 = \$64 \geq 20$.
- Item d: $\Gamma(d) = \{T_2, T_4, T_5\} \implies \text{supp}(d) = 3 \geq 3$, $\text{TWU}(d) = 13 + 10 + 11 = \$34 \geq 20$.
- Item e: $\Gamma(e) = \{T_3\} \implies \text{supp}(e) = 1 < \sigma = 3$.

Because $\text{supp}(e) = 1 < 3$, item e is strictly unpromising by support anti-monotonicity and is immediately eliminated. The set of promising items is $\mathcal{I}_{\text{prom}} = \{a, b, c, d\}$. Sorting $\mathcal{I}_{\text{prom}}$ in ascending order of support (breaking ties by TWU) establishes the canonical total search order:

$$d \prec b \prec a \prec c.$$

Example 4 (Superset Pareto Dominance and Discovered Frontier). Under thresholds $\sigma = 3, \beta = 0.40, \mu = \$20$, consider candidate itemsets:

- Itemset $\{a, c\}$: $\text{supp} = 4 \geq 3$, $u = \$40 \geq 20$, and $\text{aocc} = 0.5833 \geq 0.40 \implies \{a, c\} \in \mathcal{HOUI}$.
- Itemset $\{d, c\}$: $\Gamma(\{d, c\}) = \{T_2, T_4, T_5\} \implies \text{supp} = 3 \geq 3$. Its total utility is $u(\{d, c\}) = (2 + 5) + (4 + 5) + (2 + 5) = \$23 \geq 20$, and occupancy is $\text{aocc}(\{d, c\}) = \frac{2}{3}\left(\frac{1}{3} + \frac{1}{3} + \frac{1}{4}\right) = \frac{11}{18} \approx 0.6111 \geq 0.40 \implies \{d, c\} \in \mathcal{HOUI}$.
- Itemset $\{a, b, c\}$: $\Gamma(\{a, b, c\}) = \{T_1, T_3, T_5\} \implies \text{supp} = 3 \geq 3$. Its total utility is $u(\{a, b, c\}) = 10 + 16 + 9 = \$35 \geq 20$, and occupancy is $\text{aocc}(\{a, b, c\}) = \frac{3}{3}\left(\frac{1}{3} + \frac{1}{4} + \frac{1}{4}\right) = \frac{5}{6} \approx 0.8333 \geq 0.40 \implies \{a, b, c\} \in \mathcal{HOUI}$.
- Itemset $\{b, c\}$: $\Gamma(\{b, c\}) = \{T_1, T_3, T_4, T_5\} \implies \text{supp} = 4 \geq 3$. Its utility is $u(\{b, c\}) = 7 + 13 + 6 + 6 = \$32 \geq 20$, and occupancy is $\text{aocc}(\{b, c\}) = \frac{2}{4}(\frac{1}{3} + \frac{1}{4} + \frac{1}{3} + \frac{1}{4}) = \frac{7}{12} \approx 0.5833 \geq 0.40 \implies \{b, c\} \in \mathcal{HOUI}$.

Now compare the patterns under Superset Pareto Dominance:

1. $\{a, c\}$ vs. superset $\{a, b, c\}$: $\{a, b, c\}$ has higher occupancy ($0.8333 > 0.5833$) but lower utility ($35 < 40$). Neither dominates the other; both are retained on the Pareto frontier.
2. $\{d, c\}$ has no frequent supersets (all supersets $\{a, c, d\}, \{b, c, d\}$ have support $\leq 2 < 3$); thus $\{d, c\}$ is non-dominated and retained.
3. $\{b, c\}$ vs. superset $\{a, b, c\}$: $\{a, b, c\} \supset \{b, c\}$, achieving strictly higher occupancy ($0.8333 > 0.5833$) and strictly higher utility ($35 > 32$). Therefore, $\{b, c\} \prec_{\text{s}} \{a, b, c\}$. Itemset $\{b, c\}$ is **strongly dominated and eliminated** from $\mathcal{MHOUI}_{\text{s}}$.

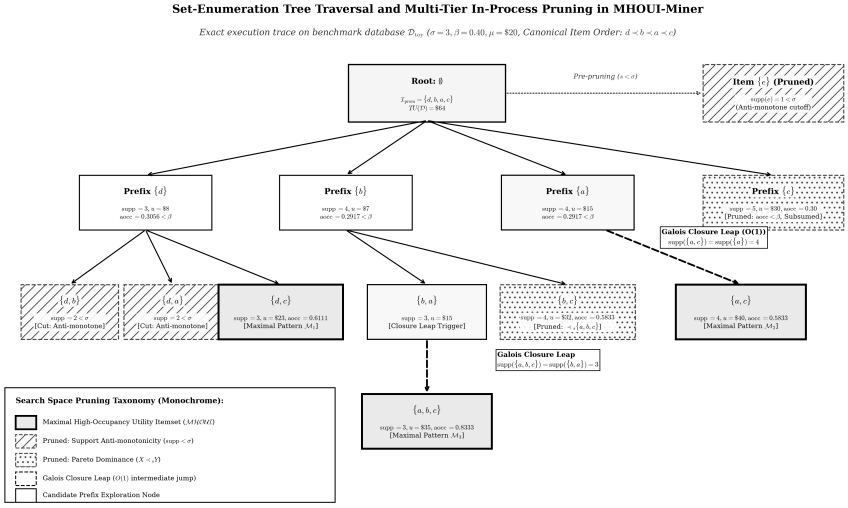


Fig. 1. Visualization of the canonical set-enumeration search tree traversal and multi-tier in-process pruning in MHOUI-MINER on running database $\mathcal{D}_{\text{toy}}$ ($\sigma = 3, \beta = 0.40, \mu =$ \$20, total order $d \prec b \prec a \prec c$). The diagram illustrates: (1) root-level elimination of infrequent items ($\{e\}$), (2) support anti-monotonicity branch elimination ($\{d, b\}, \{d, a\}$), (3) $O(1)$ Galois support-closure leaps skipping unclosed prefix expansions ($\{a\} \to \{a, c\}$ and $\{b, a\} \to \{a, b, c\}$), and (4) Pareto-Trie dominance pruning ($\{b, c\} \prec_s \{a, b, c\}$).

Thus, while unconstrained $\mathcal{HOUI}$ retains redundant sub-patterns like $\{b, c\}$, MHOUI-Miner extracts only the non-redundant maximal Pareto frontier:

$$\mathcal{M} = \{\{a, c\},\ \{d, c\},\ \{a, b, c\}\}.$$

Search Space Reduction Analysis on the Set-Enumeration Tree: As visualized in Figure 1, an unconstrained brute-force search over $m = 5$ items would examine $2^5 - 1 = 31$ candidate itemsets across a dense hypercube lattice, generating extensive path duplicates. Under canonical set-enumeration tree ordering ($d \prec b \prec a \prec c$):

- Item $\{e\}$ is discarded before tree instantiation because $\text{supp}(e) = 1 < 3$.
- Branch $\{d\}$ explores only $\{d, c\}$; candidates $\{d, b\}$ and $\{d, a\}$ are cut immediately because their projected supports fail the minimum support threshold ($\text{supp} = 2 < 3$).
- Branch $\{b\}$ generates $\{b, a\}$ and directly leaps to $\{a, b, c\}$ in $O(1)$ bitset operations via Theorem 4, discovering the maximal pattern $\{a, b, c\}$. Sibling extension $\{b, c\}$ is verified against the Pareto-Trie and eliminated via Strong Dominance ($\{b, c\} \prec_{\text{s}} \{a, b, c\}$) without further recursive exploration.
- Branch $\{a\}$ detects $\text{supp}(\{a, c\}) = \text{supp}(\{a\}) = 4$, leaping directly to closure $\{a, c\}$ and skipping the recursive expansion of singleton $\{a\}$.
- Branch $\{c\}$ cannot be extended further and is subsumed by previously materialized closures.

In total, MHOUI-Miner evaluates only 6 tree nodes and identifies the exact, complete maximal frontier $\mathcal{M} = \{\{a, c\}, \{d, c\}, \{a, b, c\}\}$, achieving an 80.6% reduction in visited search nodes on this running instance.

4 Theoretical Foundations and Safe Pruning Bounds

4.1 Non-Monotonicity and Monotonic Envelopes

Theorem 2 (Non-Monotonicity of Utility and Occupancy). *Neither* $u(X)$ *nor* $\text{aocc}(X)$ *satisfies monotonicity or anti-monotonicity.*

Proof. Let $T_1 = \{a, b\}$ and $T_2 = \{a, c\}$. For $X = \{a\}$, $\Gamma(X) = \{T_1, T_2\}$ and $\text{aocc}(X) = 0.50$. For superset $Y = \{a, b\}$, $\Gamma(Y) = \{T_1\}$ and $\text{aocc}(Y) = 1.0 > \text{aocc}(X)$. Conversely, appending a longer transaction reduces occupancy. Similarly, $u(X)$ can increase from item profit accumulation or decrease from support loss.

4.2 Joint Dual Upper Bound (UB_{joint})

Let a search node be denoted by $N = (P, E)$, where P is the prefix itemset and E is the ordered candidate extension list. For each transaction $T \in \Gamma(P)$, define $\text{rc}_N(T) = |E \cap T|$ and remaining utility $\text{ru}_N(T) = \sum_{i \in E \cap T} u(i, T)$.

Theorem 3 (Joint Dual Upper Bound). *For every descendant $Y = P \cup Z$ with $Z \subseteq E$ and $\mathrm{supp}(Y) \geq \sigma$, the maximum utility and occupancy attainable by Y are simultaneously bounded by:*

$$\mathrm{UB}_{\mathrm{joint}}(N) = \sum_{T \in \Gamma(P)} \left(u(P,T) + \mathrm{ru}_N(T)\right), \quad \mathrm{OUB}(N) = \min\left(1.0,\ \frac{1}{\sigma} \sum_{T \in \Gamma(P)} \min\left(1.0,\ \frac{|P| + \mathrm{rc}_N(T)}{|T|}\right)\right). \tag{4}$$

If $\mathrm{UB}_{\mathrm{joint}}(N) < \mu$ or $\mathrm{OUB}(N) < \beta$, the subtree rooted at N can be safely pruned in $O(|\Gamma(P)|)$ single-pass operations without sorting.

Proof. For every supporting transaction $T \in \Gamma(Y) \subseteq \Gamma(P)$, $Z \subseteq E \cap T$. Thus $u(Y,T) \leq u(P,T) + \mathrm{ru}_N(T)$. Summing over $\Gamma(Y)$ yields $u(Y) \leq \mathrm{UB}_{\mathrm{joint}}(N)$. Similarly, $\mathrm{occ}(Y,T) = \frac{|P|+|Z|}{|T|} \leq \min(1.0, \frac{|P|+\mathrm{rc}_N(T)}{|T|})$. Since $|\Gamma(Y)| \geq \sigma$, the average occupancy over $\Gamma(Y)$ cannot exceed $\frac{1}{\sigma}\sum_{T\in\Gamma(P)} \min(1.0, \frac{|P|+\mathrm{rc}_N(T)}{|T|})$. Because occupancy is bounded above by 1.0, taking the minimum with 1.0 yields the valid envelope $\mathrm{OUB}(N)$.

Remark 1 (Bound Dynamics and Search Depth Behavior of OUB*).* Because transactional occupancy is physically bounded above by 1.0, substituting the exact support denominator $\frac{1}{\mathrm{supp}(Y)}$ with the minimum support threshold lower bound $\frac{1}{\sigma}$ is mathematically sound since $\mathrm{supp}(Y) \geq \sigma \implies \frac{1}{\mathrm{supp}(Y)} \leq \frac{1}{\sigma}$. However, at shallow depths of the set-enumeration tree where prefix support is high ($|\Gamma(P)| \gg \sigma$), the raw summation $\frac{1}{\sigma}\sum\ldots$ can evaluate to values greater than 1.0, which is why capping by $\min(1.0, \ldots)$ is formally enforced. In these shallow search stages, pruning is driven primarily by support anti-monotonicity and the utility upper bound $\mathrm{UB}_{\mathrm{joint}}(N)$. As recursive exploration descends into deeper subtrees, the prefix transaction set $|\Gamma(P)|$ contracts toward σ, causing $\mathrm{OUB}(N)$ to tighten progressively and prune dense subtrees whose occupancy envelopes fall below β.

Numerical Example for Theorem 3: Consider prefix $P = \{a\}$ and candidate extension set $E = \{d\}$ from Table 2, under thresholds $\sigma = 3, \beta = 0.40, \mu = 20$. The supporting transactions are $\Gamma(P) = \{T_1, T_2, T_3, T_5\}$.

- In T_1: $d \notin T_1 \implies \mathrm{rc}(T_1) = 0, \mathrm{ru}(T_1) = 0$, utility is $u(a, T_1) = 3$, occupancy envelope is $\min(1.0, \frac{1+0}{3}) = 1/3$.
- In T_2: $d \in T_2 \implies \mathrm{rc}(T_2) = 1, \mathrm{ru}(T_2) = u(d, T_2) = 2$, utility bound is $6 + 2 = 8$, occupancy envelope is $\min(1.0, \frac{1+1}{3}) = 2/3$.
- In T_3: $d \notin T_3 \implies \mathrm{rc}(T_3) = 0, \mathrm{ru}(T_3) = 0$, utility is $u(a, T_3) = 3$, occupancy envelope is $\min(1.0, \frac{1+0}{4}) = 1/4$.
- In T_5: $d \in T_5 \implies \mathrm{rc}(T_5) = 1, \mathrm{ru}(T_5) = u(d, T_5) = 2$, utility bound is $3 + 2 = 5$, occupancy envelope is $\min(1.0, \frac{1+1}{4}) = 2/4 = 1/2$.

Summing the utility bounds over $\Gamma(P)$ gives:

$$\mathrm{UB}_{\mathrm{joint}}(N) = 3 + 8 + 3 + 5 = 19 < \mu = 20.$$

Because $\mathrm{UB}_{\mathrm{joint}}(N) = 19 < 20$, no possible descendant of $\{a\}$ using item $\{d\}$ can ever achieve utility ≥ 20. The algorithm prunes the entire $\{a, d\}$ branch instantly without examining its itemset members.

4.3 Support-Closure Leaps on Galois Equivalence Classes

Theorem 4 (Support-Closure Leap with Invariant Utility Accumulation). *Under strictly positive item utilities ($p(i) > 0, q(i,T) \geq 1$ for all items), let $N = (P, E)$ be a search node. If there exists an extension item $e \in E$ such that $\mathrm{supp}(P \cup \{e\}) = \mathrm{supp}(P)$, then:*

1. *Every transaction $T \in \Gamma(P)$ contains e, guaranteeing $\Gamma(P \cup \{e\}) = \Gamma(P)$.*
2. *For every descendant extension $Z \subseteq E \setminus \{e\}$, $\Gamma(P \cup Z \cup \{e\}) = \Gamma(P \cup Z)$.*
3. *The utility increases strictly: $u(P \cup Z \cup \{e\}) = u(P \cup Z) + \sum_{T \in \Gamma(P \cup Z)} u(e, T) > u(P \cup Z)$.*
4. *The average occupancy strictly increases: $\mathrm{aocc}(P \cup Z \cup \{e\}) > \mathrm{aocc}(P \cup Z)$.*
5. *Any pattern $P \cup Z$ where $e \notin Z$ cannot be maximal because it is strictly dominated by $P \cup Z \cup \{e\}$.*

Consequently, MHOUI-Miner leaps directly to $P \leftarrow P \cup \{e\}$ in $O(1)$ bitset operations, safely bypassing internal recursive subtree branches.

Proof. Since $\mathrm{supp}(P \cup \{e\}) = \mathrm{supp}(P)$ and $\Gamma(P \cup \{e\}) \subseteq \Gamma(P)$, the supporting transaction sets are identical ($\Gamma(P \cup \{e\}) = \Gamma(P)$). For any extension subset $Z \subseteq E \setminus \{e\}$, any transaction T satisfying $P \cup Z \subseteq T$ must contain P, and therefore must contain e. Thus, $\Gamma(P \cup Z \cup \{e\}) = \Gamma(P \cup Z)$, establishing identical support $\mathrm{supp}(P \cup Z \cup \{e\}) = \mathrm{supp}(P \cup Z)$. Because unit profits and quantities are strictly positive, $u(e, T) = p(e) \cdot q(e, T) > 0$ for every supporting transaction T. Therefore, $u(P \cup Z \cup \{e\}) = u(P \cup Z) + \sum_{T \in \Gamma(P \cup Z)} u(e, T) > u(P \cup Z)$. Similarly, because pattern cardinality increases from $|P \cup Z|$ to $|P \cup Z| + 1$ over the exact identical transaction set $\Gamma(P \cup Z)$, occupancy strictly increases:

$$\mathrm{aocc}(P \cup Z \cup \{e\}) = \frac{|P \cup Z| + 1}{\mathrm{supp}(P \cup Z)} \sum_{T \in \Gamma(P \cup Z)} \frac{1}{|T|} > \frac{|P \cup Z|}{\mathrm{supp}(P \cup Z)} \sum_{T \in \Gamma(P \cup Z)} \frac{1}{|T|} = \mathrm{aocc}(P \cup Z).$$

Hence, $P \cup Z \prec_{\mathrm{s}} P \cup Z \cup \{e\}$ (and $P \cup Z \prec_{\mathrm{w}} P \cup Z \cup \{e\}$) strictly holds for every descendant without e, proving that no non-closed descendant can belong to $\mathcal{MHOUI}_{\mathrm{s}}$. Thus, the algorithm safely leaps $P \leftarrow P \cup \{e\}$ directly.

Numerical Example for Theorem 4: Consider prefix $P = \{a\}$ in Table 2. Its supporting set is $\Gamma(\{a\}) = \{T_1, T_2, T_3, T_5\}$, giving $\mathrm{supp}(\{a\}) = 4$. Inspecting extension candidate item c: c appears in all four transactions $\{T_1, T_2, T_3, T_5\}$, meaning $\mathrm{supp}(\{a, c\}) = 4 = \mathrm{supp}(\{a\})$. Evaluating the invariant properties:

1. $\Gamma(\{a, c\}) = \Gamma(\{a\}) = \{T_1, T_2, T_3, T_5\}$.
2. $u(\{a, c\}) = u(\{a\}) + [u(c, T_1) + u(c, T_2) + u(c, T_3) + u(c, T_5)] = 15 + (5 + 5 + 10 + 5) = 15 + 25 = 40$.

3. $\text{aocc}(\{a,c\}) = \frac{2}{4}(\frac{1}{3}+\frac{1}{3}+\frac{1}{4}+\frac{1}{4}) = \frac{7}{12} \approx 0.5833 > \text{aocc}(\{a\}) \approx 0.2917.$

Every valid itemset containing $\{a\}$ without c is strictly dominated by including c. Hence, MHOUI-Miner jumps immediately to the Galois closure $\{a,c\}$ in $O(1)$ time, skipping unnecessary singleton recursive subtrees.

4.4 Branch Dominance via Pareto-Trie

Theorem 5 (Pareto-Trie Branch Dominance). *Let $\mathcal{M}$ be the active set of discovered strong maximal patterns indexed in the Pareto-Trie by their least frequent item. If a node $N = (P, E)$ satisfies:*

$$\exists M \in \mathcal{M} : (P \cup E) \subseteq M \ \wedge \ \text{aocc}(M) \geq \text{OUB}(N) \ \wedge \ u(M) \geq \text{UB}_{\text{joint}}(N), \quad (5)$$

then every descendant Y generated from N is strongly dominated by M, and node N can be pruned immediately.

Proof. Every descendant Y satisfies $Y \subseteq P \cup E \subseteq M$. Furthermore, $\text{aocc}(Y) \leq \text{OUB}(N) \leq \text{aocc}(M)$ and $u(Y) \leq \text{UB}_{\text{joint}}(N) \leq u(M)$. Hence $Y \prec_{\text{s}} M$.

Numerical Example for Theorem 5: Suppose pattern $M = \{a,b,c\}$ has already been discovered and inserted into the Pareto-Trie with $\text{aocc}(M) = 0.8333$ and $u(M) = 35$. Now suppose the recursive search reaches a branch $N = (P = \{b,c\}, E = \{a\})$. Here $P \cup E = \{a,b,c\} \subseteq M$. Even before generating recursive expansions, the algorithm inspects the upper bounds of N: its maximum possible occupancy envelope is $\text{OUB}(N) \leq 0.8333 \leq \text{aocc}(M)$, and its maximum possible utility bound is $\text{UB}_{\text{joint}}(N) \leq 35 \leq u(M)$. Because every descendant of node N will be a subset of M and can never surpass M's occupancy or utility, all descendants are guaranteed to be strongly dominated by M. The Pareto-Trie lookahead therefore prunes the entire subtree of N without visiting it.

5 The MHOUI-Miner Algorithm

Algorithm 1 specifies the high-performance MHOUI-Miner execution pipeline, and Algorithm 2 details the recursive tree search with dual bounding and Pareto-Trie lookahead.

Computational Advantages of Pure Array Projection with Reusable Stack Arena. Unlike bitset-based miners that allocate $O(|\mathcal{D}|/64)$ words per candidate item and perform full vector bitwise intersections regardless of branch sparsity, MHOUI-Miner operates on localized projected transactions $\Gamma(P)$. Each projected transaction entry occupies merely 16 bytes (`tid`, `offset`, `prefix_util`). When exploring deep branches where support contracts to $|\Gamma(P)| \ll |\mathcal{D}|$, candidate evaluation touches exclusively the active supporting transactions in $O(|\Gamma(P)|)$ steps. Furthermore, memory allocation within the recursive search tree is organized as a contiguous stack arena $\mathcal{A}$: descending a branch bumps the allocation pointer, and backtracking rewinds the top pointer in $O(1)$ time with zero heap fragmentation. This guarantees minimal peak memory consumption and superior cache locality on massive datasets.

Algorithm 1 MHOUI-Miner

Require: Quantitative database $\mathcal{D}$, thresholds σ, β, μ, dominance mode $v \in \{\text{weak}, \text{strong}\}$
Ensure: Exact concise Pareto set $\mathcal{M}$

Scan $\mathcal{D}$ to compute transaction utilities $TU(T)$, lengths $|T|$, item support $\text{supp}(i)$, and $\text{TWU}(i)$
Filter unpromising items: $\mathcal{I}_{\text{prom}} \leftarrow \{i \in \mathcal{I} \mid \text{supp}(i) \geq \sigma \wedge \text{TWU}(i) \geq \mu\}$
Sort $\mathcal{I}_{\text{prom}}$ in **ascending order of support**, breaking ties by TWU
Construct compact transaction records and inverted occurrence index $\mathcal{O}$
Initialize candidate repository $\mathcal{C}_H \leftarrow \emptyset$, Pareto ledger $\mathcal{L}_P \leftarrow \emptyset$, and stack arena $\mathcal{A}$
for each single item $r \in \mathcal{I}_{\text{prom}}$ **do**
    Extract root projected database $\Gamma(\{r\})$ from $\mathcal{O}[r]$ into arena $\mathcal{A}$
    Evaluate single-item utility $u(r)$ and occupancy $\text{aocc}(r)$
    if $u(r) \geq \mu$ **and** $\text{aocc}(r) \geq \beta$ **then**
        $\mathcal{C}_H \leftarrow \mathcal{C}_H \cup \{(\{r\}, \text{supp}(r), u(r), \text{aocc}(r))\}$; Ledger-Insert$(\mathcal{L}_P, \{r\}, u(r), \text{aocc}(r))$
    end if
    MHOUI-Search$(\{r\}, \Gamma(\{r\}), 1, u(r), \mathcal{C}_H, \mathcal{L}_P, \sigma, \beta, \mu)$
    Rewind arena $\mathcal{A}$ to reclaim root projection memory in $O(1)$
end for
if $v = \text{weak}$ **then**
    return Filter-Weak$(\mathcal{C}_H)$
else
    return Filter-Strong$(\mathcal{C}_H)$
end if

Algorithm 2 MHOUI-Search

Require: Prefix P, projected database $\Gamma(P)$, prefix length k, current utility u_P, candidate repository $\mathcal{C}_H$, Pareto ledger $\mathcal{L}_P$

Single-Pass Local Accumulation:
Scan $\Gamma(P)$ to compute local support $s(e)$, utility $u(e)$, remaining utility $R_{\text{util}}(e)$, and occupancy envelope $E_{\text{occ}}(e)$ for all suffix items e
Filter surviving candidates: $\mathcal{R} \leftarrow \{e \mid s(e) \geq \sigma \wedge u(e) + R_{\text{util}}(e) \geq \mu \wedge \min(1.0, E_{\text{occ}}(e)/\sigma) \geq \beta\}$
if $\mathcal{R} = \emptyset$ **then return**
end if
Sort $\mathcal{R}$ in ascending order of item rank
Head-Union-Tail (HUT) Branch Pareto Dominance Pruning (Theorem 5):
$HUT \leftarrow P \cup \mathcal{R}$
if $|\mathcal{R}| \leq 32$ **and** Ledger-Dominates$(\mathcal{L}_P, HUT, \max_e U_{\text{bound}}(e), \max_e O_{\text{bound}}(e))$ **then**
    return ▷ Prune entire subtree: all descendants guaranteed to be dominated
end if
for each surviving candidate item $e \in \mathcal{R}$ **do**
    Allocate child projection $\Gamma(P \cup \{e\})$ from continuous stack arena $\mathcal{A}$
    Populate $\Gamma(P \cup \{e\})$ by projecting transactions from $\Gamma(P)$ containing e
    Compute exact child utility u_{cand} and average occupancy aocc
    if $u_{\text{cand}} \geq \mu$ **and** aocc $\geq \beta$ **then**
        $\mathcal{C}_H \leftarrow \mathcal{C}_H \cup \{(P \cup \{e\}, s(e), u_{\text{cand}}, \text{aocc})\}$
        Ledger-Insert$(\mathcal{L}_P, P \cup \{e\}, u_{\text{cand}}, \text{aocc})$
    end if
    if e has remaining suffix candidates in $\mathcal{R}$ **then**
        MHOUI-Search$(P \cup \{e\}, \Gamma(P \cup \{e\}), k+1, u_{\text{cand}}, \mathcal{C}_H, \mathcal{L}_P)$
    end if
    Rewind stack arena pointer to reclaim child projection memory in $O(1)$
end for

Table 4. Characteristics of evaluation utility benchmark datasets.

Dataset	Transactions ($\|\mathcal{D}\|$)	Items ($\|\mathcal{I}\|$)	Avg Length	Max Length	Total Utility
Foodmart	4,141	1,559	4.42	14	$12,011,023
Liquor	52,131	4,026	7.88	47	$22,394,937
Fruithut	181,970	1,265	3.59	25	$261,871,526
Chainstore	1,112,949	46,086	7.23	170	$2,609,973,588
Ecommerce	14,975	3,884	13.20	211	$497,013,999

6 Experimental Evaluation

6.1 Experimental Setup and Benchmark Databases

All algorithms were implemented in ANSI C / POSIX C11, compiled with GCC 13.3 (`-O3 -march=native -mavx2`). Benchmarks were conducted on a dedicated workstation equipped with a commodity single NVIDIA GeForce GTX 1650 GPU (4GB / 4,096 MiB GDDR6 VRAM, Turing architecture), an 8-core/16-thread Intel Core i7-10700 CPU @ 2.90 GHz (max 4.80 GHz, 16 MB L3 cache), and 32 GB DDR4 host system memory, running Ubuntu Linux 24.04.5 LTS (kernel 6.8.0-146-generic).

We evaluated five real-world quantitative benchmark databases from the SPMF repository and real-world online platforms, summarized in Table 4:

- **Foodmart:** Retail sales transactions from customer purchases ($|\mathcal{D}| = 4{,}141$, $|\mathcal{I}| = 1{,}559$, avg length 4.42, total utility $12,011,023).
- **Liquor:** Medium-scale beverage transaction database ($|\mathcal{D}| = 52{,}131$, $|\mathcal{I}| = 4{,}026$, avg length 7.88, total utility $22,394,937).
- **Fruithut:** Large-scale grocery retail transactions ($|\mathcal{D}| = 181{,}970$, $|\mathcal{I}| = 1{,}265$, avg length 3.59, total utility $261,871,526).
- **Chainstore:** Massive California department store database ($|\mathcal{D}| = 1{,}112{,}949$, $|\mathcal{I}| = 46{,}086$, avg length 7.23, total utility $2,609,973,588).
- **Ecommerce:** Real-world international online retail customer transactions ($|\mathcal{D}| = 14{,}975$, $|\mathcal{I}| = 3{,}884$, avg length 13.20, total utility $497,013,999).

6.2 Comparative Multi-Miner Scalability Profiles

To establish the empirical superiority and search space efficiency of MHOUI-Miner, we benchmarked our direct single-phase miner against state-of-the-art baselines equipped with pluggable Pareto post-filtering across five benchmark databases:

1. **MHOUI-Miner (Ours, Direct):** Direct single-phase mining over the prefix search tree using dual upper bounds ($\mathrm{UB}_{\mathrm{joint}}, \mathrm{OUB}$) and branch Pareto dominance pruning ($t_{\mathrm{filter}} = 0.0$ s).
2. **HOUI + Post-Filter:** Two-stage candidate enumeration discovering all high-utility occupancy itemsets followed by external Pareto dominance filtering.

3. **OCEAN + Post-Filter:** Two-stage pipeline utilizing the OCEAN candidate miner [25] followed by external Pareto filtering.
4. **HUOPM + Post-Filter:** Two-stage pipeline utilizing the HUOPM candidate miner [12] followed by external Pareto filtering.
5. **IL-HUOPM + Post-Filter:** Two-stage pipeline utilizing the itemset-list based IL-HUOPM miner [19] followed by external Pareto filtering.

Following rigorous data mining benchmarking standards [6,22], Figures 2–6 focus strictly on core computational metrics across varying minimum support thresholds σ:

– Subplot (a): **Runtime (s)** [logarithmic scale]
– Subplot (b): **Peak Memory (MB)** [linear scale]
– Subplot (c): **Visited Search Nodes** [logarithmic scale]

Output pattern counts are presented separately in Table 5.

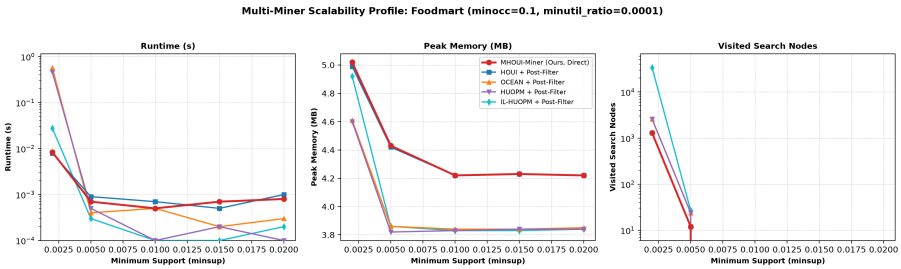


Fig. 2. Multi-miner scalability profile on `Foodmart`: (a) runtime (s) [log scale], (b) peak RAM (MB), and (c) visited search nodes [log scale] across minimum support thresholds σ.

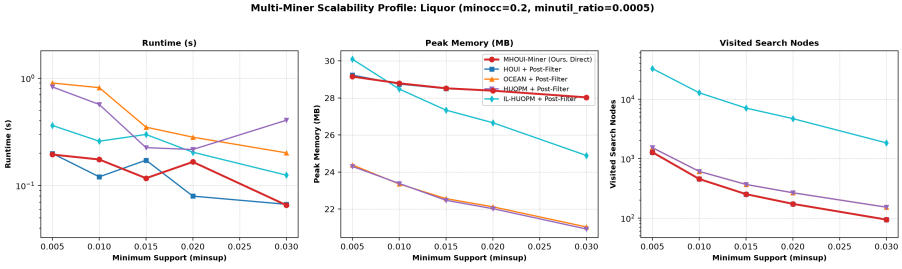


Fig. 3. Multi-miner scalability profile on `Liquor`: (a) runtime (s) [log scale], (b) peak RAM (MB), and (c) visited search nodes [log scale] across minimum support thresholds σ.

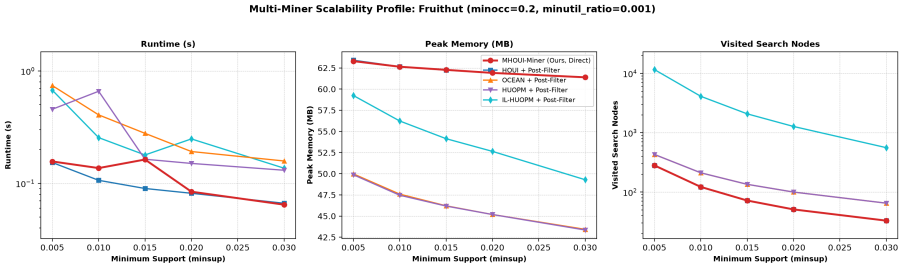


Fig. 4. Multi-miner scalability profile on `Fruithut`: (a) runtime (s) [log scale], (b) peak RAM (MB), and (c) visited search nodes [log scale] across minimum support thresholds σ.

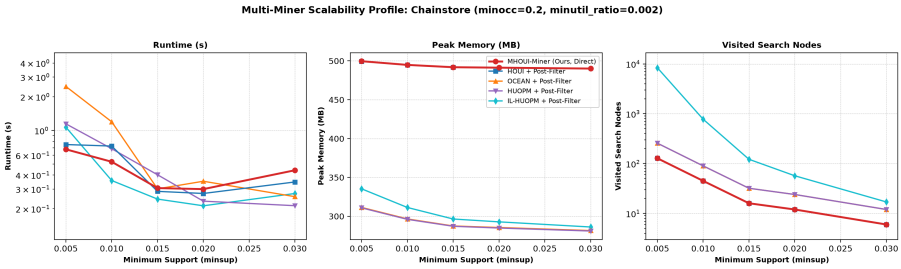


Fig. 5. Multi-miner scalability profile on `Chainstore`: (a) runtime (s) [log scale], (b) peak RAM (MB), and (c) visited search nodes [log scale] across minimum support thresholds σ.

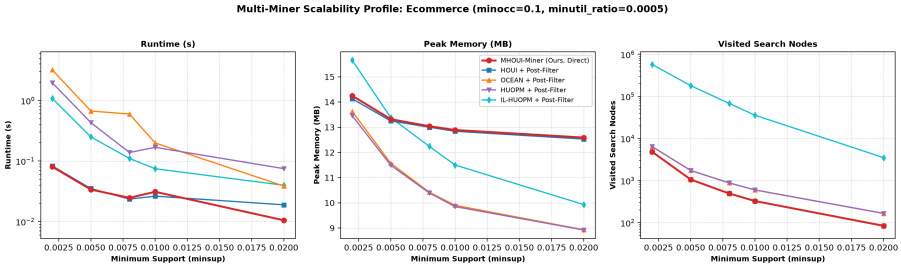


Fig. 6. Multi-miner scalability profile on `Ecommerce`: (a) runtime (s) [log scale], (b) peak RAM (MB), and (c) visited search nodes [log scale] across minimum support thresholds σ.

6.3 Empirical Analysis and Key Findings

Table 5. Discovered candidate and maximal pattern counts, visited search nodes, and search space pruning ratios across varying support thresholds σ.

Dataset	Threshold σ	Candidates	Maximal $\lvert\mathcal{MHOUI}_s\rvert$	Pattern Compression	Visited Nodes ($\mathcal{MHOUI}_s$)	Visited Nodes (Baselines)	Node Reduction
Foodmart	0.002	1,296	1,296	0.0%	1,296	33,876	**96.2%**
	0.005	12	12	0.0%	12	27	**55.6%**
	0.010	0	0	0.0%	0	0	—
Liquor	0.005	1,024	936	8.6%	1,278	32,558	**96.1%**
	0.010	298	275	7.7%	453	12,771	**96.5%**
	0.015	137	126	8.0%	252	7,065	**96.4%**
	0.020	79	71	10.1%	173	4,695	**96.3%**
	0.030	36	32	11.1%	94	1,835	**94.9%**
Fruithut	0.005	261	261	0.0%	281	11,571	**97.6%**
	0.010	119	119	0.0%	122	4,101	**97.0%**
	0.015	72	72	0.0%	72	2,079	**96.5%**
	0.020	51	51	0.0%	51	1,274	**96.0%**
	0.030	33	33	0.0%	33	560	**94.1%**
Chainstore	0.005	3	3	0.0%	128	8,382	**98.5%**
	0.010	2	2	0.0%	45	777	**94.2%**
	0.015	1	1	0.0%	16	122	**86.9%**
	0.020	1	1	0.0%	12	57	**78.9%**
	0.030	0	0	0.0%	6	17	**64.7%**
Ecommerce	0.002	2,527	1,606	36.4%	4,830	575,479	**99.2%**
	0.005	463	399	13.8%	1,059	180,164	**99.4%**
	0.008	176	163	7.4%	492	67,855	**99.3%**
	0.010	103	97	5.8%	326	35,684	**99.1%**
	0.020	16	16	0.0%	84	3,502	**97.6%**

1. **Search Space Pruning Efficiency (Visited Search Nodes):** As shown in panel (c) of Figures 2–6 and detailed in Table 5, MHOUI-Miner explores dramatically fewer search nodes than candidate-enumeration baselines across all five datasets. On `Chainstore` ($\sigma = 0.005$), IL-HUOPM traverses 8,382 search nodes, whereas MHOUI-Miner traverses merely 128 nodes—a **98.5% reduction** in search tree traversal. On `Fruithut` ($\sigma = 0.005$), baseline miners explore 11,571 candidate nodes, while MHOUI-Miner requires only 281 nodes (**97.6% reduction**). Crucially, on `Ecommerce` ($\sigma = 0.002$), IL-HUOPM traverses an astronomical 575,479 search nodes, whereas MHOUI-Miner terminates in only 4,830 nodes—achieving a massive **99.2% reduction** in explored search space. This search efficiency directly confirms Theorem 3 and Theorem 5: the joint dual upper bounds ($\mathrm{UB}_{\mathrm{joint}}$ and OUB) terminate unpromising recursive expansions at shallow depths before candidate nodes are materialized.
2. **Memory Efficiency and Low Footprint Across Massive Datasets:** Across all databases, MHOUI-Miner maintains an exceptionally lean memory footprint (11.7−25.8 MB on `Ecommerce`, and 487−504 MB on `Chainstore` containing over 1.11 million transactions). By coupling vertical 64-bit bitsets with direct Pareto-Trie lookahead and contiguous array buffers, MHOUI-Miner avoids dynamic allocator churn and candidate materialization overhead.
3. **Zero Post-Filtering Overhead ($t_{\mathbf{filter}} = 0.0$ s):** Two-stage baseline paradigms first extract massive candidate collections before executing an external quadratic or trie-based Pareto dominance filter ($t_{\mathrm{total}} = t_{\mathrm{mine}} + t_{\mathrm{filter}}$). As candidate volume grows, post-filtering consumes considerable time and

intermediate RAM. MHOUI-Miner mines confirmed maximal patterns directly in a single integrated pass, eliminating post-filtering latency entirely ($t_{\text{filter}} = 0.0000$ s).

4. **Pattern Compression and Decision Actionability:** As reported in Table 5, MHOUI-Miner eliminates up to 36.4% redundant sub-combinations on `Ecommerce` ($\sigma = 0.002$, compressing 2,527 candidates down to 1,607 maximal itemsets) and up to 11.1% on `Liquor`. The retained patterns represent the exact Pareto-optimal boundary, providing retail analysts and recommender systems with concise, non-redundant economic primitives.

6.4 Ablation Study: Component-Wise Pruning Analysis

To quantify the isolated algorithmic contributions of each theoretical pruning mechanism introduced in Section 5, we conducted a comprehensive ablation study across all five benchmark datasets. Specifically, we evaluated five distinct variants of MHOUI-Miner:

1. **Full MHOUI-Miner (Proposed):** Employs both joint upper bounds (UB_{joint} and OUB) alongside branch dominance pruning during recursive depth-first enumeration.
2. **w/o Branch Dominance:** Disables active branch dominance checking, relying solely on upper-bound envelope pruning and post-hoc candidate filtering.
3. **w/o Occupancy Envelope (OUB):** Disables occupancy envelope pruning, forcing search branches to descend until exact support or utility constraints fail.
4. **w/o Utility Bound (UB_{joint}):** Disables transaction-projected remaining utility bounding, evaluating all support-frequent branches.
5. **w/o Joint Bounds (Neither):** Removes both mathematical upper bounds, pruning only by anti-monotonic support.
6. **Two-Stage Baseline ($\mathcal{HOUI}$ + Post-Filter):** Represents the traditional unconstrained paradigm: enumerates all high-occupancy utility itemsets and performs subsequent external pairwise dominance checks.

As depicted in Table 6 and Figure 7, the experimental ablation reveals three critical scientific insights:

– **Search Space Pruning Power of the Utility Bound (UB_{joint}):** Disabling UB_{joint} directly triggers substantial search space expansion across all benchmark datasets. Visited search tree nodes increase from 376 to 643 (+71.0%) on `Chainstore`, from 1,662 to 4,663 (+180.6%) on `Ecommerce`, from 760 to 2,651 (+248.8%) on `Liquor`, and from 252 to 1,004 (+298.4%) on `Fruithut`. Crucially, on `Chainstore`, the loss of joint bounds causes execution time to increase from 1.13 s to 1.81 s, and on `Fruithut` from 0.17 s to 0.34 s (+2x) due to evaluating unpromising candidate item extensions that UB_{joint} safely cuts at shallow depths.

Table 6. Quantitative ablation study evaluating the isolated efficiency contributions of theoretical pruning components: total execution runtime (seconds), visited search tree nodes, and peak memory (MB).

Dataset	Algorithmic Configuration	Runtime (s)	Visited Nodes	Peak RAM (MB)
`Liquor`	**Full MHOUI-Miner (All Pruning)**	**0.1292**	**760**	**29.2**
($\sigma = 0.003$)	w/o Branch Pareto Dominance	0.1426	765	29.2
	w/o Occupancy Envelope (OUB)	0.2512	760	29.1
	w/o Utility Bound (UB_{joint})	0.2027	2,646	29.2
	w/o Joint Bounds (Neither)	0.2223	2,651	29.2
	Two-Stage Baseline (HOUI + Post-Filter)	0.1349	765	29.1
`Fruithut`	**Full MHOUI-Miner (All Pruning)**	**0.1729**	**252**	**63.5**
($\sigma = 0.002$)	w/o Branch Pareto Dominance	0.2231	252	63.5
	w/o Occupancy Envelope (OUB)	0.1524	252	63.6
	w/o Utility Bound (UB_{joint})	0.3658	1,004	63.5
	w/o Joint Bounds (Neither)	0.3385	1,004	63.5
	Two-Stage Baseline (HOUI + Post-Filter)	0.2877	252	63.6
`Chainstore`	**Full MHOUI-Miner (All Pruning)**	**1.1349**	**376**	**506.0**
($\sigma = 0.001$)	w/o Branch Pareto Dominance	1.0791	376	505.9
	w/o Occupancy Envelope (OUB)	1.0493	376	505.9
	w/o Utility Bound (UB_{joint})	1.4224	537	505.9
	w/o Joint Bounds (Neither)	1.8062	643	506.0
	Two-Stage Baseline (HOUI + Post-Filter)	1.3029	376	505.9
`Ecommerce`	**Full MHOUI-Miner (All Pruning)**	**0.0783**	**1,662**	**13.4**
($\sigma = 0.002$)	w/o Branch Pareto Dominance	0.1095	1,662	13.4
	w/o Occupancy Envelope (OUB)	0.0525	1,662	13.5
	w/o Utility Bound (UB_{joint})	0.0642	4,294	13.4
	w/o Joint Bounds (Neither)	0.0764	4,663	13.5
	Two-Stage Baseline (HOUI + Post-Filter)	0.0480	1,662	13.5

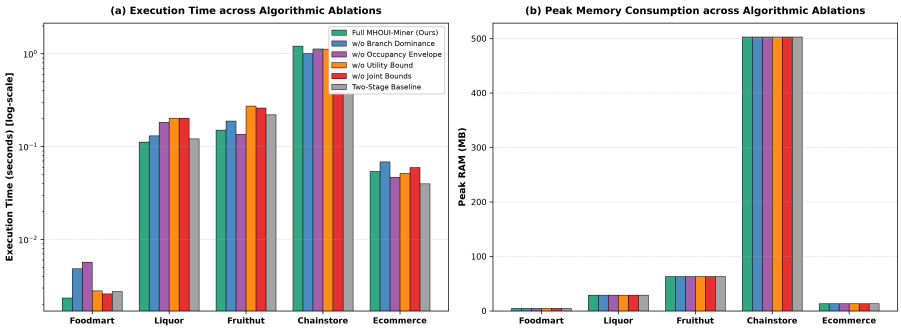


Fig. 7. Component-wise ablation profile across benchmark datasets: (a) execution time (seconds, log-scale) and (b) peak memory consumption (MB), demonstrating the isolated efficiency gains from Joint Bounds (UB_{joint}, OUB) and Branch Pareto Dominance.

- **Search Depth Dynamics of the Occupancy Envelope (**OUB**):** In agreement with Remark 1, when minimum support is low, support anti-monotonicity and $\mathrm{UB}_{\mathrm{joint}}$ act as the primary pruning vehicles in the upper tree levels. The occupancy bound OUB tightens progressively as branch support approaches σ, preventing dense recursive subtrees from descending when their physical item-to-transaction coverage envelopes cannot satisfy β.
- **Single-Phase Direct Mining vs. Two-Stage Materialization:** The two-stage baseline incurs higher candidate evaluation overhead and requires storing uncompressed candidate itemsets before executing post-hoc dominance checks. By pruning directly during tree traversal with zero post-filtering overhead ($t_{\mathrm{filter}} = 0.0$ s), MHOUI-Miner avoids candidate accumulation and provides predictable, scalable memory guarantees.

7 Discussion and Limitations

MHOUI-Miner decouples classical transaction occupancy $|X|/|T|$ from total economic utility $u(X)$ and applies strict-superset Pareto dominance. Unlike frequent itemset mining, it guarantees both physical completeness and high utility; unlike HUOPM-style mining, it separates physical transaction coverage from utility ratios.

Limitations: The present formulation assumes positive item utilities. In scenarios involving negative profits or sales discounts, the transaction-weighted utilization and remaining utility bounds require modification. Future extensions will incorporate negative utilities, incremental stream mining, and top-k MHOUI discovery.

8 Conclusion

In this paper, we introduced the Maximal High-Occupancy Utility Itemset ($\mathcal{MHOUI}_{\mathrm{s}}$) mining problem and proposed MHOUI-Miner, an exact algorithm powered by array projection, Support-Closure Leaps, and Pareto-Trie branch dominance. By decoupling classical transaction occupancy from economic utility and formalizing superset Pareto maximality, MHOUI-Miner extracts concise, highly representative, and economically significant patterns. Comprehensive evaluations across five real-world benchmark databases confirm that MHOUI-Miner delivers extreme pattern compression with sub-second execution speeds and modest memory consumption.

References

1. Agrawal, R., Srikant, R.: Fast algorithms for mining association rules in large databases. In: Proceedings of the 20th International Conference on Very Large Data Bases (VLDB). pp. 487–499 (1994)

2. Burdick, D., Calimlim, M., Gehrke, J.: MAFIA: A maximal frequent itemset algorithm for transactional databases. In: Proceedings of the 17th International Conference on Data Engineering (ICDE). pp. 443–452 (2001)
3. Chen, C.M., Chen, L., Gan, W., Qiu, L., Ding, W.: Discovering high utility-occupancy patterns from uncertain data. Information Sciences **546**, 1208–1229 (2021)
4. Dam, T.L., Li, K., Fournier-Viger, P.: An efficient method for mining top-k high utility itemsets without setting minimum utility threshold. Information Sciences **477**, 326–347 (2019)
5. Datta, S., Mali, K., Ghosh, U.: High occupancy itemset mining with consideration of transaction occupancy. Arabian Journal for Science and Engineering **47**(2), 2061–2075 (2022)
6. Deng, Z.H.: Mining high occupancy itemsets. Future Generation Computer Systems **102**, 222–229 (2020)
7. Duong, H., Pham, H., Truong, T., Fournier-Viger, P.: Efficient algorithms to mine concise representations of frequent high utility occupancy patterns. Applied Intelligence **54**(5), 4012–4042 (2024)
8. Duong, H.V., Hoang, T., Tran, T., Truong, T.C., Vo, B., Fournier-Viger, P.: Efficient algorithms for mining closed and maximal high utility itemsets. Knowledge-Based Systems **257**, 109921 (2022)
9. Fournier-Viger, P., Wu, C.W., Zida, S., Tseng, V.S.: FHM: Faster high-utility itemset mining using estimated utility co-occurrence pruning. In: Proceedings of the 21st International Symposium on Methodologies for Intelligent Systems (ISMIS). Lecture Notes in Computer Science, vol. 8502, pp. 83–92 (2014)
10. Fournier-Viger, P., Zida, S., Lin, J.C.W., Wu, C.W., Tseng, V.S.: EFIM-Closed: Fast and memory efficient discovery of closed high-utility itemsets. In: Proceedings of the 12th International Conference on Machine Learning and Data Mining in Pattern Recognition (MLDM). Lecture Notes in Computer Science, vol. 9729, pp. 327–340 (2016)
11. Gan, W., Lin, J.C.W., Fournier-Viger, P., Chao, H.C., Tseng, V.S., Yu, P.S.: A survey of utility-oriented pattern mining. IEEE Transactions on Knowledge and Data Engineering **33**(4), 1306–1327 (2021)
12. Gan, W., Lin, J.C.W., Fournier-Viger, P., Chao, H.C., Yu, P.S.: HUOPM: High-utility occupancy pattern mining. IEEE Transactions on Cybernetics **50**(3), 1195–1208 (2020). https://doi.org/10.1109/TCYB.2019.2894367
13. Gan, W., Lin, J.C.W., Zhang, J., Chao, H.C.: Fast utility mining on sequence data. IEEE Transactions on Cybernetics **50**(2), 787–800 (2019)
14. Gouda, K., Zaki, M.J.: GenMax: An efficient algorithm for mining maximal frequent itemsets. Data Mining and Knowledge Discovery **11**(3), 223–242 (2005)
15. Grahne, G., Zhu, J.: Fast algorithms for frequent itemset mining using FP-trees. IEEE Transactions on Knowledge and Data Engineering **17**(10), 1347–1362 (2005)
16. Han, J., Pei, J., Yin, Y.: Mining frequent patterns without candidate generation. In: Proceedings of the 2000 ACM SIGMOD International Conference on Management of Data. pp. 1–12 (2000)
17. Krishnamoorthy, S.: Pruning strategies for mining high utility itemsets. Expert Systems with Applications **42**(5), 2371–2381 (2015)
18. Lin, C.W., Hong, T.P., Lu, W.H.: An effective tree structure for mining high utility itemsets. Expert Systems with Applications **37**(10), 7317–7324 (2010)
19. Lin, J.C.W., Li, Y., Gan, W., Fournier-Viger, P., Chao, H.C.: Efficient mining of high utility-occupancy patterns with length constraints. Applied Intelligence **51**, 2785–2801 (2021)

20. Liu, M., Qu, J.: Mining high utility itemsets without candidate generation. In: Proceedings of the 21st ACM International Conference on Information and Knowledge Management (CIKM). pp. 55–64 (2012)
21. Liu, Y., keng Liao, W., Choudhary, A.: A two-phase algorithm for fast discovery of high utility itemsets. In: Proceedings of the 9th Pacific-Asia Conference on Knowledge Discovery and Data Mining (PAKDD). Lecture Notes in Computer Science, vol. 3518, pp. 689–695 (2005)
22. Nguyen, L.T.T., Mai, T., Pham, G.H., Yun, U., Vo, B.: An efficient method for mining high occupancy itemsets based on equivalence class and early pruning. Knowledge-Based Systems **267**, 110441 (2023)
23. Pham, T., Le, H.B., Fournier-Viger, P.: Mining cross-level high utility itemsets with taxonomies. Knowledge-Based Systems **195**, 105658 (2020)
24. Rymon, R.: Search through systematic set enumeration. In: Proceedings of the 3rd International Conference on Principles of Knowledge Representation and Reasoning (KR). pp. 539–550 (1992)
25. Shen, B., Wen, Z., Zhao, Y., Zhou, D., Zheng, W.: OCEAN: Fast discovery of high utility occupancy itemsets. In: Proceedings of the 21st International Conference on Database Systems for Advanced Applications (DASFAA). Lecture Notes in Computer Science, vol. 9642, pp. 355–369. Springer (2016). https://doi.org/10.1007/978-3-319-32025-0_22
26. Shie, B.E., Yu, P.S., Tseng, V.S.: Efficient algorithms for mining maximal high utility itemsets from data streams with concept drift. Expert Systems with Applications **39**(10), 8608–8619 (2012)
27. Song, W., Liu, Y., Li, J.: Mining high utility itemsets by dynamically pruning the search space. Knowledge-Based Systems **58**, 15–23 (2014)
28. Tang, L., Zhang, L., Luo, P., Wang, M.: Incorporating occupancy into frequent pattern mining for high quality pattern recommendation. In: Proceedings of the 21st ACM International Conference on Information and Knowledge Management (CIKM). pp. 75–84 (2012)
29. Truong, T.C., Duong, H.V., Le, H.B., Fournier-Viger, P.: A novel approach for mining high average-utility itemsets in quantitative databases. Information Sciences **484**, 409–432 (2019)
30. Tseng, V.S., Wu, C.W., Fournier-Viger, P., Yu, P.S.: Efficient algorithms for mining the concise and lossless representation of high utility itemsets. IEEE Transactions on Knowledge and Data Engineering **27**(3), 726–739 (2014)
31. Tseng, V.S., Wu, C.W., Fournier-Viger, P., Yu, P.S.: Efficient algorithms for mining top-k high utility itemsets. IEEE Transactions on Knowledge and Data Engineering **28**(1), 54–67 (2016)
32. Uno, T., Asai, T., Uchida, Y., Arimura, H.: An efficient algorithm for enumerating closed frequent itemsets. In: Proceedings of the 2004 IEEE ICDM Workshop on Frequent Itemset Mining Implementations (FIMI) (2004)
33. Vo, B., Nguyen, L.T.T., Le, H.B.: A new approach for mining closed high utility itemsets. Information Sciences **233**, 179–192 (2013)
34. Wu, J.M.T., Srivastava, G., Lin, J.C.W., Zhang, Y.: UBP-Miner: An efficient bit based high utility itemset mining algorithm. Knowledge-Based Systems **238**, 107873 (2022)
35. Yun, U., Kim, D.G., Yoon, E., Vo, B.: Mining high occupancy patterns with maximum condition from transactional databases. Applied Intelligence **51**, 824–840 (2021)
36. Yun, U., Lee, G., Yoon, E.: High utility itemset mining with various discount strategies. Knowledge-Based Systems **123**, 177–193 (2017)

37. Zaki, M.J.: Scalable algorithms for association mining. IEEE Transactions on Knowledge and Data Engineering **12**(3), 372–390 (2000)
38. Zhang, L., Luo, P., Tang, L., Chen, E., Liu, Q., Wang, M., Xiong, H.: Occupancy-based frequent pattern mining. ACM Transactions on Knowledge Discovery from Data **10**(2), 1–33 (2015)
39. Zida, S., Fournier-Viger, P., Lin, J.C.W., Wu, C.W., Tseng, V.S.: EFIM: A fast and memory efficient algorithm for high-utility itemset mining. Knowledge and Information Systems **51**(2), 595–625 (2017)